

Concurrency : User-Directed Waiting with Conditions

Lecture 11

Professor Amanda Bienz

What if threads need to wait intelligently?

- Consider the queue example from last time

```
33     int Queue_Dequeue(queue_t *q, void *value) {
34         pthread_mutex_lock(&q->headLock);
35         node_t *tmp = q->head;
36         node_t *newHead = tmp->next;
37         if (newHead == NULL) {
38             pthread_mutex_unlock(&q->headLock);
39             return -1; // queue was empty
40         }
41         *value = newHead->value;
42         q->head = newHead;
43         pthread_mutex_unlock(&q->headLock);
44         free(tmp);
45         return 0;
46     }
```

- Fails if the queue is empty
- What do we want to happen if the queue is empty?

What if threads need to wait intelligently?

- What if we checked newHead until it was not empty?

```
33     int Queue_Dequeue(queue_t *q, void *value) {
34         pthread_mutex_lock(&q->headLock);
35         node_t *tmp = q->head;
36         node_t *newHead = tmp->next;
37         while (newHead == NULL) {
38             pthread_mutex_unlock(&q->headLock);
39             // Let other threads run
40             pthread_mutex_lock(&q->headlock);
41             tmp = q->head; newHead = tmp->next;
42         }
43         *value = newHead->value;
44         q->head = newHead;
45         pthread_mutex_unlock(&q->headLock);
46         free(tmp);
47         return 0;
48     }
```

- Does this work? Is it a good idea?

What if threads need to wait intelligently?

- What if we checked newHead until it was not empty?

```
33     int Queue_Dequeue(queue_t *q, void *value) {
34         pthread_mutex_lock(&q->headLock);
35         node_t *tmp = q->head;
36         node_t *newHead = tmp->next;
37         while (newHead == NULL) {
38             pthread_mutex_unlock(&q->headLock);
39             // Let other threads run
40             pthread_mutex_lock(&q->headlock);
41             tmp = q->head; newHead = tmp->next;
42         }
43         *value = newHead->value;
44         q->head = newHead;
45         pthread_mutex_unlock(&q->headLock);
46         free(tmp);
47         return 0;
48     }
```

- Does this work? Is it a good idea?
- **We want to wait until newHead is non-NULL, not spin**

Condition Variables

- There are many cases where a thread wishes to check whether a condition is true before continuing its execution
- Simpler examples:
 - A parent thread might wish to check whether a child thread has completed
 - This is often called a `join()`
 - Similar cases when a thread wants to wait on other threads to reach a particular execution point (e.g. all threads have started)
 - This is often called a `barrier()`

Condition Variables

- Condition variables are useful when some kind of **signaling** must take place between threads

```
int pthread_cond_wait(pthread_cond_t *cond,  
                      pthread_mutex_t *mutex);  
int pthread_cond_signal(pthread_cond_t *cond);
```

- pthread_cond_wait :
 - Put the calling thread to sleep
 - Wait for some other thread to signal it
- pthread_cond_signal :
 - Unblock at least one of the threads that are blocked on the condition variable

How to wait for a condition

- Condition variable
 - Waiting on the condition
 - An explicit queue that threads can put themselves in when some state of execution is not as desired
 - Signaling on the condition
 - Some other thread, when it changes said state, can wake one of those waiting threads and allow them to continue

Parent waiting for Child : Use a condition variable

- Parent :
 - Create the child thread and continues running itself
 - Call into `thr_join()` to wait for the child thread to complete
 - Acquire the lock, check if child is done, put itself to sleep (`wait()`), release the lock
- Child :
 - Print the message “child”
 - Call `thr_exit()` to wake the parent thread
 - Grab the lock, set the state variable *done*, signal the parent thus waking it

Attempt 1

```
1      void thr_exit() {
2          pthread_mutex_lock(&m);
3          pthread_cond_signal(&c);
4          pthread_mutex_unlock(&m);
5      }
6
7      void thr_join() {
8          pthread_mutex_lock(&m);
9          pthread_cond_wait(&c, &m);
10         pthread_mutex_unlock(&m);
11     }
```

- Imagine the case where the child runs immediately
 - The child will signal, but there is no thread asleep on the condition
 - When the parent runs, it will call wait and be stuck
 - **No thread will ever wake it**

Attempt 2

```
1      void thr_exit() {
2          done = 1;
3          Pthread_cond_signal(&c);
4      }
5
6      void thr_join() {
7          if (done == 0)
8              Pthread_cond_wait(&c);
9      }
```

- The issue here is a subtle **race condition**
 - The parent calls thr_join()
 - The parent checks the value of *done*
 - It will see that it is 0 and try to go to sleep
 - Just before it calls wait to go to sleep, the parent is interrupted and the child runs
 - The child changes the state variable *done* to 1 and signals
 - But no thread is waiting and thus no thread is woken
 - When the parent runs again, it sleeps forever

Parent waiting for Child : Proper use of condition variable

```
1      int done = 0;
2      pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3      pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5      void thr_exit() {
6          Pthread_mutex_lock(&m);
7          done = 1;
8          Pthread_cond_signal(&c);
9          Pthread_mutex_unlock(&m);
10     }
11
12     void *child(void *arg) {
13         printf("child\n");
14         thr_exit();
15         return NULL;
16     }
17
18     void thr_join() {
19         Pthread_mutex_lock(&m);
20         while (done == 0)
21             Pthread_cond_wait(&c, &m);
22         Pthread_mutex_unlock(&m);
23     }
24
```

Parent waiting for Child : Proper use of condition variable

(cont.)

```
25      int main(int argc, char *argv[]) {
26          printf("parent: begin\n");
27          pthread_t p;
28          Pthread_create(&p, NULL, child, NULL);
29          thr_join();
30          printf("parent: end\n");
31          return 0;
32      }
```

Condition Variables (Cont.)

- A thread calling wait routine:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t init = PTHREAD_COND_INITIALIZER;

pthread_mutex_lock(&lock);
while (initialized == 0)
    pthread_cond_wait(&init, &lock);
pthread_mutex_unlock(&lock);
```

- The wait call releases the lock when putting the caller to sleep
- Before returning after being woken, the wait call re-acquires the lock
- A thread calling signal routing:

```
pthread_mutex_lock(&lock);
initialized = 1;
pthread_cond_signal(&init);
pthread_mutex_unlock(&lock);
```

The Producer / Consumer (Bound Buffer) Problem

- Like the queue problem we started with
- Producer :
 - Produce data items
 - Wish to place data items in a buffer
- Consumer
 - Grab data items out of the buffer, consume them in some way
- Example : Multi-threaded web server
 - A producer puts HTTP requests into a work queue
 - Consumer threads take requests out of the queue and process them

Bounded Buffer

- A bounded buffer is used when you pipe the output of one program into another
 - Example : *grep foo file.txt | wc -l*
 - The grep process is the producer
 - The wc process is the consumer
 - Between them is an in-kernel bounded buffer
- Bounded buffer is a shared resource : synchronized access is required

The Put and Get Routines (Version 1)

```
1      int buffer;  
2      int count = 0;    // initially, empty  
3  
4      void put(int value) {  
5          assert(count == 0);  
6          count = 1;  
7          buffer = value;  
8      }  
9  
10     int get() {  
11         assert(count == 1);  
12         count = 0;  
13         return buffer;  
14     }
```

- Only put data into the buffer when count is zero
 - i.e. when buffer is empty
- Only get data from buffer when count is one
 - i.e. when buffer is full

Producer/Consumer Threads (Version 1)

```
1      void *producer(void *arg) {
2          int i;
3          int loops = (int) arg;
4          for (i = 0; i < loops; i++) {
5              put(i);
6          }
7      }
8
9      void *consumer(void *arg) {
10         int i;
11         while (1) {
12             int tmp = get();
13             printf("%d\n", tmp);
14         }
15     }
```

- Producer puts an integer into the shared buffer loops number of times
- Consumer gets the data out of that shared buffer

Producer/Consumer : Single CV and If Statement

- A single condition variable cond and associated lock mutex

```
1      cond_t cond;
2      mutex_t mutex;
3
4      void *producer(void *arg) {
5          int i;
6          for (i = 0; i < loops; i++) {
7              Pthread_mutex_lock(&mutex);           // p1
8              if (count == 1)                       // p2
9                  Pthread_cond_wait(&cond, &mutex); // p3
10             put(i);                                // p4
11             Pthread_cond_signal(&cond);            // p5
12             Pthread_mutex_unlock(&mutex);          // p6
13         }
14     }
15
16     void *consumer(void *arg) {
17         int i;
18         for (i = 0; i < loops; i++) {
19             Pthread_mutex_lock(&mutex);           // c1
```

Producer/Consumer : Single CV and If Statement

```
20         if (count == 0)                                // c2
21             Pthread_cond_wait(&cond, &mutex);           // c3
22         int tmp = get();                                  // c4
23         Pthread_cond_signal(&cond);                      // c5
24         Pthread_mutex_unlock(&mutex);                    // c6
25         printf("%d\n", tmp);
26     }
27 }
```

- p1-p3 : a producer waits for the buffer to be empty
- c1-c3 : a consumer waits for the buffer to be full
- With just a single producer and a single consumer, the code works

If we have **more than** one of producer and consumer?

Thread Trace : Broken Solution (Version 1)

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep		Ready	p1	Running	0	
	Sleep		Ready	p2	Running	0	
	Sleep		Ready	p4	Running	1	Buffer now full
	Ready		Ready	p5	Running	1	T_{c1} awoken
	Ready		Ready	p6	Running	1	
	Ready		Ready	p1	Running	1	
	Ready		Ready	p2	Running	1	
	Ready		Ready	p3	Sleep	1	Buffer full; sleep
	Ready	c1	Running		Sleep	1	T_{c2} sneaks in ...
	Ready	c2	Running		Sleep	1	
	Ready	c4	Running		Sleep	0	... and grabs data
	Ready	c5	Running		Ready	0	T_p awoken
	Ready	c6	Running		Ready	0	
c4	Running		Ready		Ready	0	Oh oh! No data

Thread Trace : Broken Solution (Version 1)

- The problem arises for a simple reason:
 - After the producer woke $T_{\{c1\}}$, but before $T_{\{c1\}}$ ever ran, the state of the bounded buffer was changed by $T_{\{c2\}}$
 - There is no guarantee that when the woken thread runs, the state will still be as desired : Mesa semantics
 - Virtually every system ever built employs Mesa semantics
 - Hoare semantics provides a stronger guarantee that the woken thread will run immediately upon being woken

Producer/Consumer : Single CV and While

- Consumer T_{c1} wakes up and re-checks the state of the shared variable
 - If the buffer is empty, the consumer simply goes back to sleep

```
1      cond_t cond;
2      mutex_t mutex;
3
4      void *producer(void *arg) {
5          int i;
6          for (i = 0; i < loops; i++) {
7              Pthread_mutex_lock(&mutex);           // p1
8              while (count == 1)                   // p2
9                  Pthread_cond_wait(&cond, &mutex); // p3
10             put(i);                               // p4
11             Pthread_cond_signal(&cond);           // p5
12             Pthread_mutex_unlock(&mutex);         // p6
13         }
14     }
15
```

Condition Variables (Cont.)

- The waiting thread should always re-check the condition in a while loop, instead of a simple if statement

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t init = PTHREAD_COND_INITIALIZER;  
  
pthread_mutex_lock(&lock);  
while (initialized == 0)  
    pthread_cond_wait(&init, &lock);  
pthread_mutex_unlock(&lock);
```

- Without rechecking, the waiting thread will continue thinking that the condition has changed even though it has not

Producer/Consumer : Single CV and While

```
(Cont.)
16     void *consumer(void *arg) {
17         int i;
18         for (i = 0; i < loops; i++) {
19             Pthread_mutex_lock(&mutex);           // c1
20             while (count == 0)                    // c2
21                 Pthread_cond_wait(&cond, &mutex); // c3
22             int tmp = get();                       // c4
23             Pthread_cond_signal(&cond);           // c5
24             Pthread_mutex_unlock(&mutex);         // c6
25             printf("%d\n", tmp);
26         }
27     }
```

- A simple rule to remember with condition variables is to always use while loops
- However, this code still has a bug!

Thread Trace : Broken Solution (Version 2)

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep	c1	Running		Ready	0	
	Sleep	c2	Running		Ready	0	
	Sleep	c3	Sleep		Ready	0	Nothing to get
	Sleep		Sleep	p1	Running	0	
	Sleep		Sleep	p2	Running	0	
	Sleep		Sleep	p4	Running	1	Buffer now full
	Ready		Sleep	p5	Running	1	T_{c1} awoken
	Ready		Sleep	p6	Running	1	
	Ready		Sleep	p1	Running	1	
	Ready		Sleep	p2	Running	1	
	Ready		Sleep	p3	Sleep	1	Must sleep (full)
c2	Running		Sleep		Sleep	1	Recheck condition
c4	Running		Sleep		Sleep	0	T_{c1} grabs data
c5	Running		Ready		Sleep	0	Oops! Woke T_{c2}

Thread Trace : Broken Solution (Version 2) (Cont.)

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
...	...	...	...	...	...	...	(<i>cont.</i>)
c6	Running		Ready		Sleep	0	
c1	Running		Ready		Sleep	0	
c2	Running		Ready		Sleep	0	
c3	Sleep		Ready		Sleep	0	Nothing to get
	Sleep	c2	Running		Sleep	0	
	Sleep	c3	Sleep		Sleep	0	Everyone asleep ...

- A consumer should not wake other consumers, only producers, and vice-versa

The Single Buffer Producer/Consumer Solution

- Use two condition variables and while
 - Producer threads wait on the condition empty and signals fill
 - Consumer threads wait on fill and signal empty

```
1      cond_t empty, fill;
2      mutex_t mutex;
3
4      void *producer(void *arg) {
5          int i;
6          for (i = 0; i < loops; i++) {
7              Pthread_mutex_lock(&mutex);
8              while (count == 1)
9                  Pthread_cond_wait(&empty, &mutex);
10             put(i);
11             Pthread_cond_signal(&fill);
12             Pthread_mutex_unlock(&mutex);
13         }
14     }
15
```

The Single Buffer Producer/Consumer Solution

```
(Cont.)
16      void *consumer(void *arg) {
17          int i;
18          for (i = 0; i < loops; i++) {
19              Pthread_mutex_lock(&mutex);
20              while (count == 0)
21                  Pthread_cond_wait(&fill, &mutex);
22              int tmp = get();
23              Pthread_cond_signal(&empty);
24              Pthread_mutex_unlock(&mutex);
25              printf("%d\n", tmp);
26          }
27      }
```

The Final Producer/Consumer Solution

- More concurrency and efficiency : Add more buffer slots
 - Allow concurrent production or consuming to take place
- Reduce context switches

```
1      int buffer[MAX];
2      int fill = 0;
3      int use = 0;
4      int count = 0;
5
6      void put(int value) {
7          buffer[fill] = value;
8          fill = (fill + 1) % MAX;
9          count++;
10     }
11
12     int get() {
13         int tmp = buffer[use];
14         use = (use + 1) % MAX;
15         count--;
16         return tmp;
17     }
```

The Final Put and Get Routines

The Final Producer/Consumer Solution (Cont.)

```
1      cond_t empty, fill;
2      mutex_t mutex;
3
4      void *producer(void *arg) {
5          int i;
6          for (i = 0; i < loops; i++) {
7              Pthread_mutex_lock(&mutex);           // p1
8              while (count == MAX)                 // p2
9                  Pthread_cond_wait(&empty, &mutex); // p3
10             put(i);                               // p4
11             Pthread_cond_signal(&fill);           // p5
12             Pthread_mutex_unlock(&mutex);         // p6
13         }
14     }
15
16     void *consumer(void *arg) {
17         int i;
18         for (i = 0; i < loops; i++) {
19             Pthread_mutex_lock(&mutex);           // c1
20             while (count == 0)                    // c2
21                 Pthread_cond_wait(&fill, &mutex); // c3
22             int tmp = get();                      // c4
```

The Final Producer/Consumer Solution

(Cont.)

```
23          Pthread_cond_signal(&empty);           // c5
24          Pthread_mutex_unlock(&mutex);          // c6
25          printf("%d\n", tmp);
26      }
27 }
```

The Final Working Solution (Cont.)

- P2 : a producer only sleeps if all buffers are currently filled
- C2 : a consumer only sleeps if all buffers are currently empty

One More Example : Covering Conditions

- Assume there are zero bytes free
 - Thread T_a calls allocate(100)
 - Thread T_b calls allocate(10)
 - Both T_a and T_b wait on the condition and go to sleep
 - Thread T_c calls free(50)

Which waiting thread should be woken up?

Covering Conditions (Cont.)

```
1      // how many bytes of the heap are free?
2      int bytesLeft = MAX_HEAP_SIZE;
3
4      // need lock and condition too
5      cond_t c;
6      mutex_t m;
7
8      void *
9      allocate(int size) {
10         Pthread_mutex_lock(&m);
11         while (bytesLeft < size)
12             Pthread_cond_wait(&c, &m);
13         void *ptr = ...;           // get mem from heap
14         bytesLeft -= size;
15         Pthread_mutex_unlock(&m);
16         return ptr;
17     }
18
19     void free(void *ptr, int size) {
20         Pthread_mutex_lock(&m);
21         bytesLeft += size;
22         Pthread_cond_signal(&c);   // whom to signal??
23         Pthread_mutex_unlock(&m);
24     }
```

Covering Conditions (Cont.)

- Solution (Suggested by Lampson and Redell)
 - Replace `pthread_cond_signal()` with `pthread_cond_broadcast()`
 - `Pthread_cond_broadcast()`:
 - Wake up all waiting threads
 - Because threads have to acquire a mutex, they will leave the condition and run only one at a time
 - Cost : too many threads may be woken
 - Threads that shouldn't be awake will simply wake up, re-check the condition, and then go back to sleep